MOVIE TICKET BOOKING SYSTEM

IT23431 – MONGODB ESSENTIALS

Submitted by

CHARLES C - (241001038)

ARUN SR - (241001019)

DEEPAK O - (241001042)

OF

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project titled “Movie Ticket Booking System” is the bonafide work of “**CHARLES C (241001038), ARUN SR (241001019), DEEPAK O (241001042)**” who carried out the project work under my supervision.

SIGNATURE
Dr.Valarmathie
HEAD OF THE DEPARTMENT
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Chennai – 602105

SIGNATURE
Mrs.Usha S
COURSE INCHARGE
Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Chennai – 602105

This project is submitted for IT23431 - MONGODB ESSENTIALS held on _____

INTERNAL EXAMINAR EXTERNAL EXAMINAR

TABLE OF CONTENTS

1 MOVIE TICKET BOOKING SYSTEM

1.1 ABSTRACT

The Movie Ticket Booking System is a web-based application designed to simplify and automate the process of booking movie tickets. This system provides users with an intuitive and user-friendly interface to browse available movies, check show timings, select seats, and complete bookings efficiently. The primary objective of the project is to eliminate the traditional manual booking process and enhance user convenience through a digital platform. MongoDB is used as the database to store user information, movie details, booking records, and seat availability, ensuring efficient data management and scalability.

The system includes key features such as user registration and login, movie listing, real-time seat selection, booking confirmation, and ticket management. By integrating these technologies, the application ensures fast performance, data security, and ease of use. This project demonstrates the practical implementation of full-stack development concepts and provides a foundation for building more advanced online reservation systems in the future.

1.2 INTRODUCTION

The Movie Ticket Booking System is a web-based application designed to simplify the process of booking movie tickets online. It allows users to browse available movies, select show timings, choose seats, and make secure payments. This system eliminates the need for physical ticket counters and provides a seamless digital experience. The application is developed using Python Flask with MongoDB for backend. The system also integrates an online payment gateway to enable secure transactions.

1.3 PURPOSE

The purpose of the Movie Ticket Booking System is to provide a convenient and efficient platform for users to book movie tickets online without visiting physical ticket counters. The system is designed to simplify the entire booking process by allowing users to browse available movies, select show timings, choose seats, and complete payments through a secure interface. It aims to enhance user experience by reducing waiting time, minimizing manual errors, and offering a fast and reliable ticket booking solution.

1.4 SCOPE OF THE PROJECT

The Movie Ticket Booking System is designed to cover all essential functionalities required for an online movie ticket reservation platform. It enables users to create accounts, log in securely, browse available movies, filter and search based on preferences, select show timings, and choose seats through an interactive interface. The system also supports booking confirmation and online payment processing, providing a complete end-to-end booking experience. Additionally, users can view their booking history and manage their tickets. The project is developed to function in a controlled environment using a web-based interface and a backend database. However, it does not include real-time synchronization with actual theatre systems, large-scale deployment, or advanced features such as dynamic pricing and AI-based recommendations. The scope is focused on demonstrating core functionalities and full-stack development concepts.

1.5 SOFTWARE REQUIREMENTS OF THE PROJECT

Project Functionality

- User registration and login with secure authentication
- Movie browsing with search and filter options
- Display of movie details such as genre, duration, and language
- Show selection based on date, time, and theatre
- Interactive seat selection with real-time availability
- Ticket booking with price calculation including GST
- Online payment integration (Razorpay)
- Booking confirmation and storage in database
- Viewing and managing user booking history.

Software Requirements

- Database: MongoDB
- Frontend: HTML , CSS , JavaScript
- Backend: Python (Flask)
- Technology: Flask Framework, REST APIs
- Payment Integration: Razorpay API
- TOOLS: Flask-CORS, Bcrypt

Hardware Requirements

- Processor: Intel i3 or higher
- RAM: 4GB or higher
- Hard Disk: 500GB or higher
- OS: Windows 8, 10, 11

Assumptions and Dependencies

- Users will register and create login credentials securely.]
- The system depends on MongoDB database for storing user, movie, show, and booking data.
- Internet connection is required for accessing the system and processing payments.
- The system relies on Razorpay API for payment processing.

Functional Requirements

1. Login Module (LM)
   Users can register and log in using email and password. Passwords are securely stored using encryption. Access is granted only after verification from the database.

2. Movie Management Module (MMM)
   Displays available movies with details. Allows searching and filtering of movies.

3. Show Selection Module (SSM)
   Users can view available shows based on date and theatre. Displays show timings and seat availability.

4. Seat Booking Module (SBM)
   Users can select and book seats interactively. Seat availability is updated dynamically.

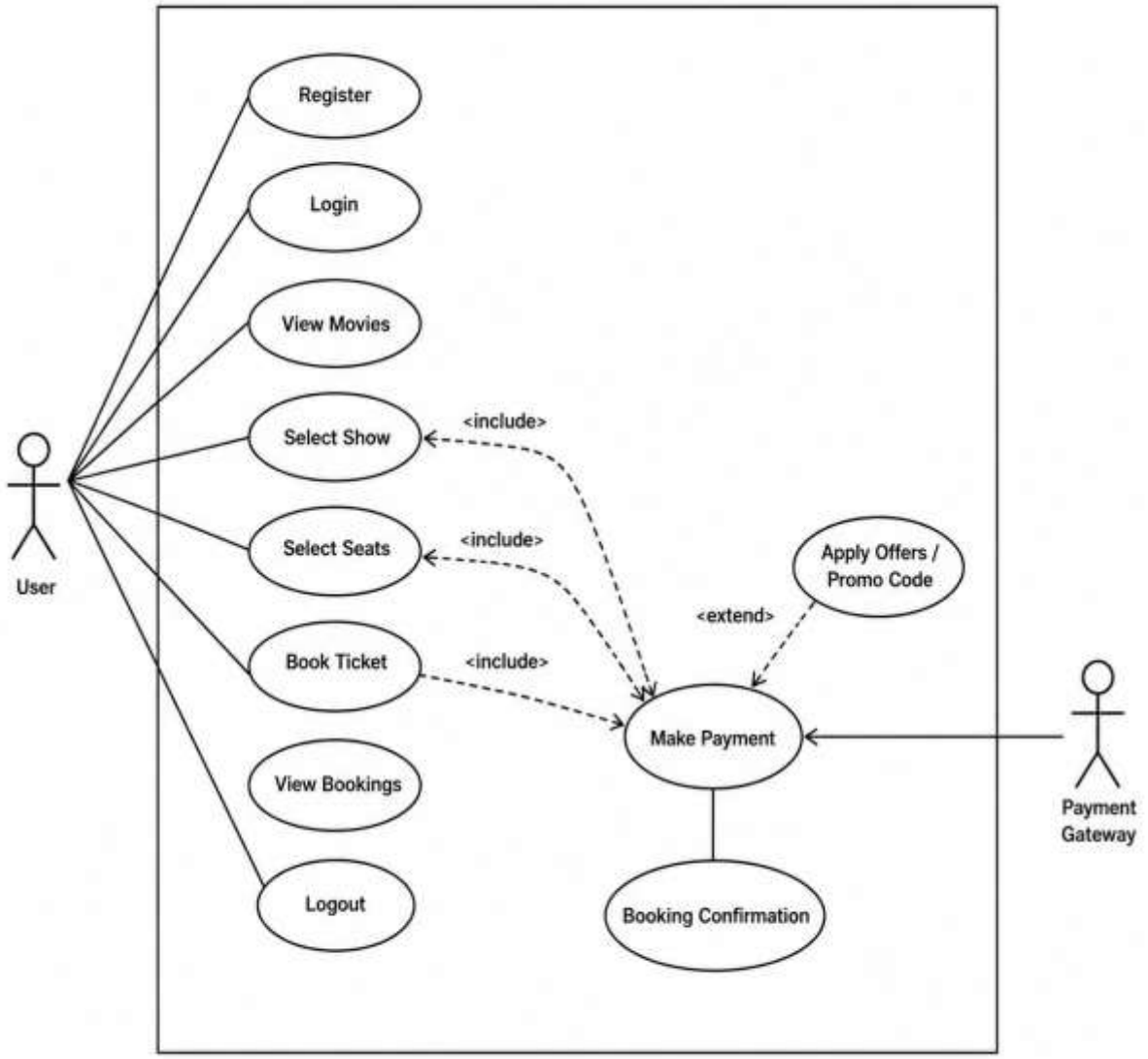
5. Payment Module (PM)
   Calculates total cost including GST and extras. Processes payments securely using Razorpay.

6. Booking Management Module (BMM)
   Stores booking details in the database. Allows users to view and manage their bookings.

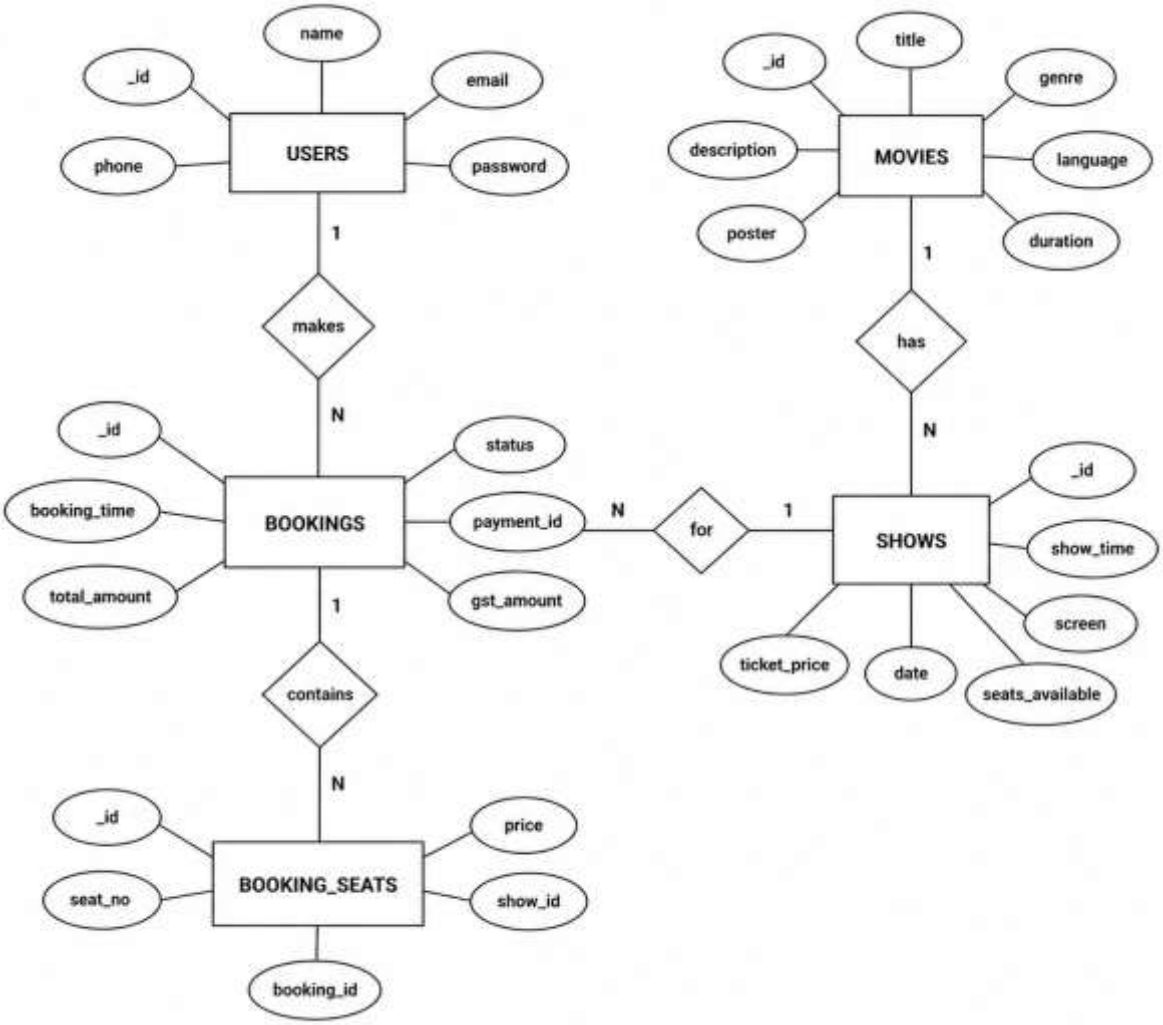
7. Server Module (SM)
   Handles communication between frontend and database. Validates requests and ensures data consistency.

2 SYSTEM DIAGRAMS

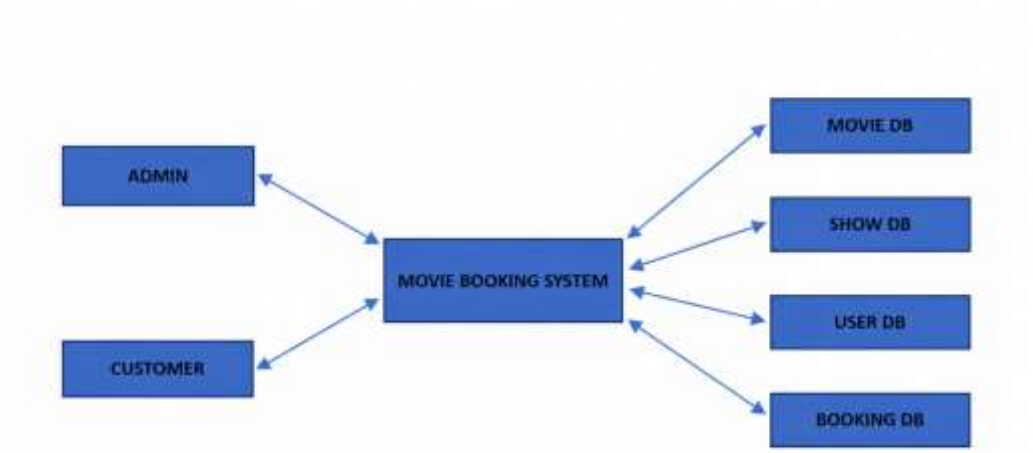
2.1 USE CASE DIAGRAM



2.2 ENTITY RELATIONSHIP DIAGRAM



2.3 DATAFLOW DIAGRAM



3 MODULE DESCRIPTION

USER LOGIN

User can login with email and password created during registration. The system verifies the credentials and allows access only after successful authentication.

USER REGISTRATION

New users can create an account by providing basic details such as name, email, and password. The details are stored securely in the database.

VIEW MOVIES

Users can view the list of available movies along with details such as genre, duration, language, and description. They can also search and filter movies.

SELECT SHOW

Users can select movie shows based on date, time, and theatre. The system displays available show timings and screens for the selected movie.

SELECT SEATS

Users can choose seats from the available seat layout. The system shows booked and available seats and updates selection dynamically.

BOOK TICKET

After selecting seats, users can proceed to book tickets. The system calculates the total amount including ticket price and additional charges.

PAYMENT

Users can make payment through the integrated payment gateway. The system processes the payment securely and verifies the transaction.

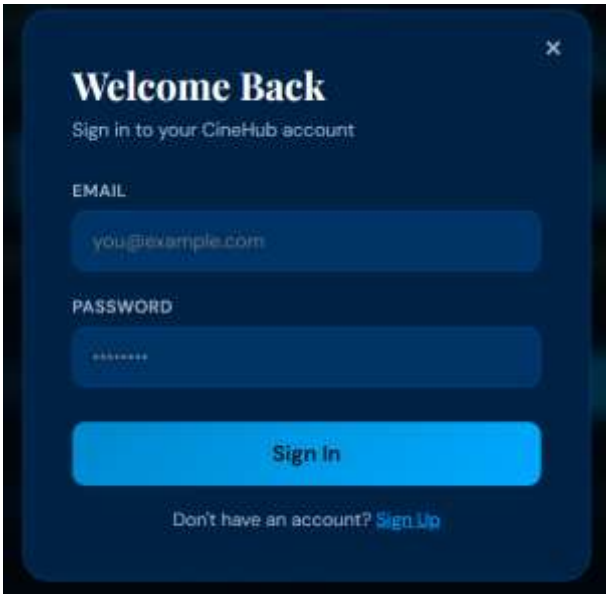
BOOKING CONFIRMATION

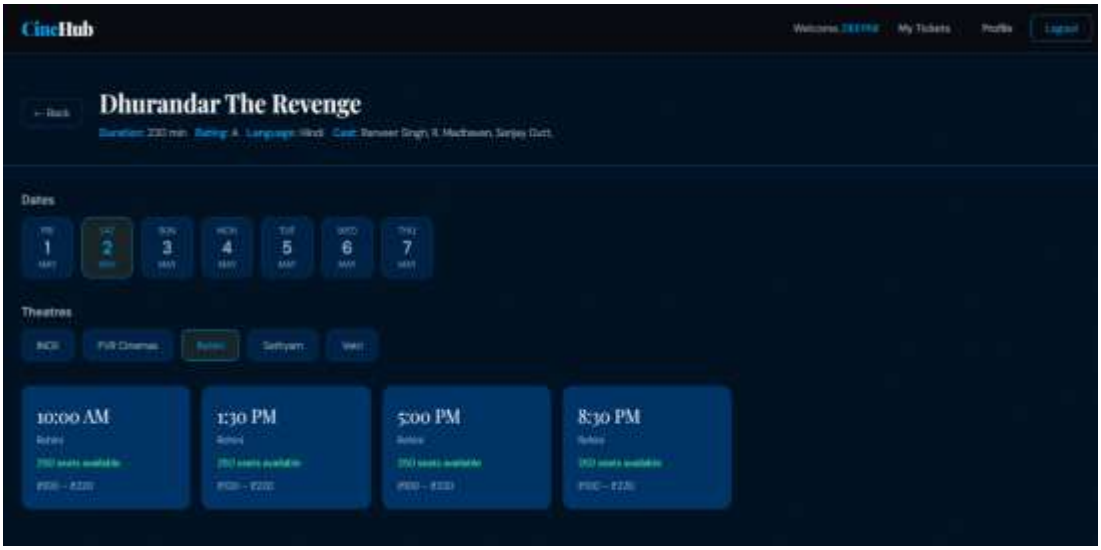
Once payment is successful, the booking is confirmed and stored in the database. The system generates booking details for the user.

VIEW BOOKINGS

Users can view their booking history, including movie details, show time, seats, and payment status.

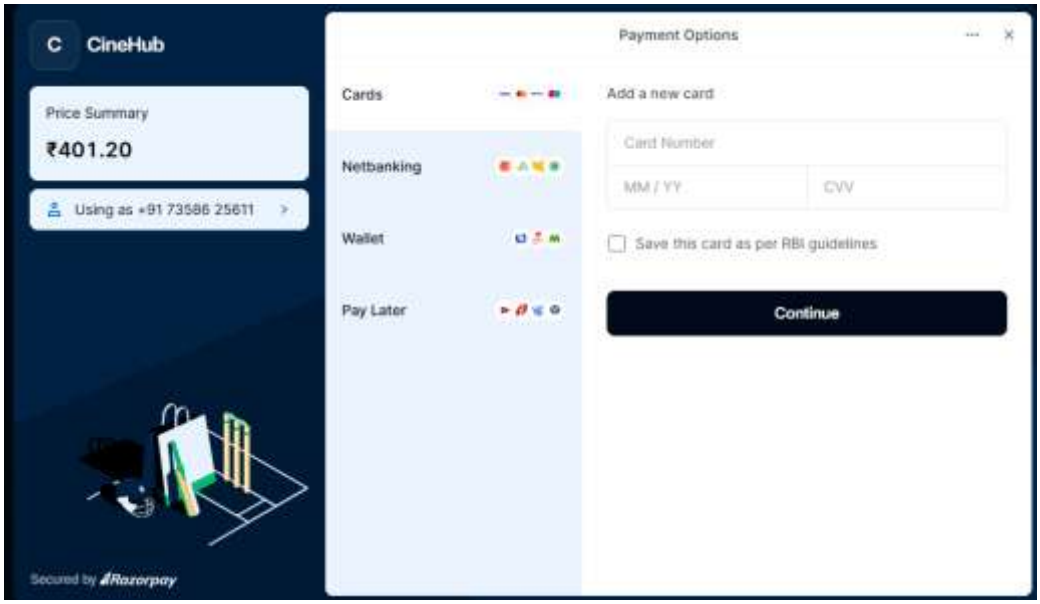
4 IMPLEMENTATION






Select Your Seats


Booking Summary
Seat F12 (Standard)
₹220
Seat F13 (Standard)
₹220
Discount
-₹100
Add Popcorn & Coke Combo
+₹250
CINE100
Apply
₹100 discount applied!
Total
₹340
Confirm 2 Seats — ₹340


CineHub
Price Summary
₹401.20
Using as +91 73586 25611
Payment Options
Cards
Netbanking
Wallet
Pay Later
Add a new card
Card Number
MM / YY
CVV
Save this card as per RBI guidelines
Continue
Secured by Razorpay

DATABASE DESIGN

The Movie Ticket Booking System uses MongoDB as its database, which is a NoSQL document-oriented database. The data is stored in collections such as users, movies, shows, and bookings. The users collection stores user details like name, email, and encrypted password. The movies collection contains information such as title, genre, duration, language, and description. The shows collection stores show-specific data including movie ID, date, time, screen, and seat availability. The bookings collection maintains records of user bookings, including selected seats, total amount, payment status, and booking time. MongoDB provides flexibility in handling dynamic data structures and ensures efficient storage and retrieval of booking information.

CODE

DATABASE CONNECTION

```
try:
    import razorpay
except ImportError:
    razorpay = None

# — Razorpay Setup ————————————————————————————————
RAZORPAY_KEY_ID = os.environ.get("RAZORPAY_KEY_ID", "rzp_test_SjM1knQI4LyBRz")
RAZORPAY_KEY_SECRET = os.environ.get("RAZORPAY_KEY_SECRET", "G74w62LLEWFojMmGWnUEQYA1")

razorpay_client = None
try:
    razorpay_client = razorpay.Client(auth=(RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET))
except Exception as e:
    print(f"⚠️ Razorpay Client Init Failed: {e}")

# — App Setup ————————————————————————————————————
app = Flask(__name__, static_folder="static", static_url_path="")
app.secret_key = os.environ.get("SECRET_KEY", "cinevault_dev_secret_2026")
app.config["SESSION_COOKIE_SAMESITE"] = "Lax"
app.config["SESSION_COOKIE_SECURE"]   = False
app.config["SESSION_COOKIE_HTTPONLY"] = True

CORS(app,
     supports_credentials=True,
     origins=["http://localhost:5000", "http://127.0.0.1:5000"])

# — MongoDB ——————————————————————————————————————
MONGO_URI = os.environ.get("MONGO_URI", "mongodb://localhost:27017/")
try:
    client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
    client.server_info()
    print("✅ Connected to MongoDB")
except Exception as e:
    print(f"❌ MongoDB connection failed: {e}")
    raise SystemExit(1)

db           = client["movie_booking"]
users_col    = db["users"]
movies_col   = db["movies"]
shows_col    = db["shows"]
bookings_col = db["bookings"]
```

REGISTER

```python
@app.route("/api/register", methods=["POST"])
def register():
    data     = request.get_json(silent=True) or {}
    name     = (data.get("name")     or "").strip()
    email    = (data.get("email")    or "").strip().lower()
    password = (data.get("password") or "").strip()
    if not name or not email or not password:
        return jsonify({"error": "All fields are required"}), 400

    # Validate password strength
    is_valid, error_msg = validate_password(password)
    if not is_valid:
        return jsonify({"error": error_msg}), 400

    if users_col.find_one({"email": email}):
        return jsonify({"error": "Email already registered"}), 409
    # FIXED: store hash as decoded string so it round-trips through MongoDB cleanly
    hashed = bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")
    uid = users_col.insert_one({
        "name": name, "email": email, "password": hashed,
        "created_at": datetime.datetime.now(datetime.UTC)
    }).inserted_id
    session["user_id"] = str(uid)
    return jsonify({"message": "Registered successfully", "name": name}), 201

@app.route("/api/login", methods=["POST"])
def login():
    data     = request.get_json(silent=True) or {}
    email    = (data.get("email")    or "").strip().lower()
    password = (data.get("password") or "").strip()
    user = users_col.find_one({"email": email})
    if not user:
        return jsonify({"error": "Invalid email or password"}), 401
    stored = user["password"]
    if isinstance(stored, str):
        stored = stored.encode("utf-8")   # handle both old bytes and new string
    if not bcrypt.checkpw(password.encode("utf-8"), stored):
        return jsonify({"error": "Invalid email or password"}), 401
    session["user_id"] = str(user["_id"])
    return jsonify({"message": "Login successful", "name": user["name"]}), 200

@app.route("/api/logout", methods=["POST"])
def logout():
    session.clear()
    return jsonify({"message": "Logged out"}), 200
```

BOOKING SEATS

```python
# ── Booking routes ──────────────────────────────────────────────
@app.route("/api/bookings", methods=["POST"])
@require_login
def create_booking():
    data    = request.get_json(silent=True) or {}
    show_id = data.get("show_id","")
    seats   = data.get("seats", [])
    snacks_price = data.get("snacks_price", 0)
    discount = data.get("discount", 0)

    if not show_id or not seats:
        return jsonify({"error": "show_id and seats required"}), 400

    try:
        show = shows_col.find_one({"_id": ObjectId(show_id)})
    except:
        return jsonify({"error": "Invalid show_id"}), 400

    if not show:
        return jsonify({"error": "Show not found"}), 404

    # 🚫 DO NOT LOCK SEATS HERE
    seat_map = {s["id"]: s for s in show["seats"]}
    total = 0

    for sid in seats:
        s = seat_map.get(sid)
        if not s:
            return jsonify({"error": f"Seat {sid} not found"}), 400
        total += s["price"]

    total += snacks_price
    total -= discount
    if total < 0:
        total = 0

    gst = round(total * 0.18, 2)
    total_with_gst = total + gst

    user = current_user()
    movie = movies_col.find_one({"_id": show["movie_id"]})

    bid = bookings_col.insert_one({
        "user_id": user["_id"],
        "user_name": user["name"],
        "user_email": user["email"],
        "show_id": ObjectId(show_id),
        "movie_id": show["movie_id"],
        "movie_title": movie["title"] if movie else "Unknown",
        "show_date": show["date"],
        "show_time": show["time"],
        "screen": show["screen"],
        "seats": seats,   # store only IDs
        "snacks_price": snacks_price,
```

PAYMENT

```python
# — PAYMENT (UPDATED CORE LOGIC) ————————————————————————————

@app.route("/api/bookings/<booking_id>/pay", methods=["POST"])
@require_login
def confirm_payment(booking_id):
    data = request.get_json(silent=True) or {}

    user = current_user()

    try:
        booking = bookings_col.find_one({
            "_id": ObjectId(booking_id),
            "user_id": user["_id"]
        })
    except:
        return jsonify({"error": "Invalid ID"}), 400

    if not booking:
        return jsonify({"error": "Booking not found"}), 404

    if booking["status"] == "confirmed":
        return jsonify({"error": "Already paid"}), 400

    if booking["status"] == "cancelled":
        return jsonify({"error": "Booking cancelled"}), 400

    # Verify Razorpay Signature (if provided and keys are real)
    payment_id = data.get("payment_id")
    order_id = data.get("order_id")
    signature = data.get("signature")

    if razorpay_client and not RAZORPAY_KEY_ID.startswith("rzp_test_dummy") and payment_id and order_id and signature:
        try:
            razorpay_client.utility.verify_payment_signature({
                'razorpay_order_id': order_id,
                'razorpay_payment_id': payment_id,
                'razorpay_signature': signature
            })
        except razorpay.errors.SignatureVerificationError:
            return jsonify({"error": "Payment verification failed"}), 400

    # 🎯 STEP 1: CHECK SEATS AGAIN (CRITICAL)
    show = shows_col.find_one({"_id": booking["show_id"]})
    seat_map = {s["id"]: s for s in show["seats"]}

    for sid in booking["seats"]:
        if seat_map[sid]["booked"]:
            return jsonify({
                "error": f"Seat {sid} already booked by another user"
            }), 409

```

5 CONCLUSION

The system provides a user-friendly interface for browsing movies, selecting shows, booking seats, and making secure payments. It integrates frontend and backend functionalities effectively and ensures proper data handling using MongoDB. The project highlights important concepts such as database management, API integration, authentication, and payment processing. Overall, the system improves the efficiency of ticket booking and serves as a practical example of real-world application development.

6 REFERENCES

- W3Schools – HTML, CSS and JavaScript Tutorials
- GeeksforGeeks – Python Flask and MongoDB Integration
- Razorpay API Documentation